

DELTA · V

Control Plane Observability

Where Horizon ended. Where you might come in.

Prepared for DJ Gregor

v1.2.0-beta2 just shipped
13 Spring Boot daemons. No Karaf. No JMX MBeans.

Why this deck

Where you left off

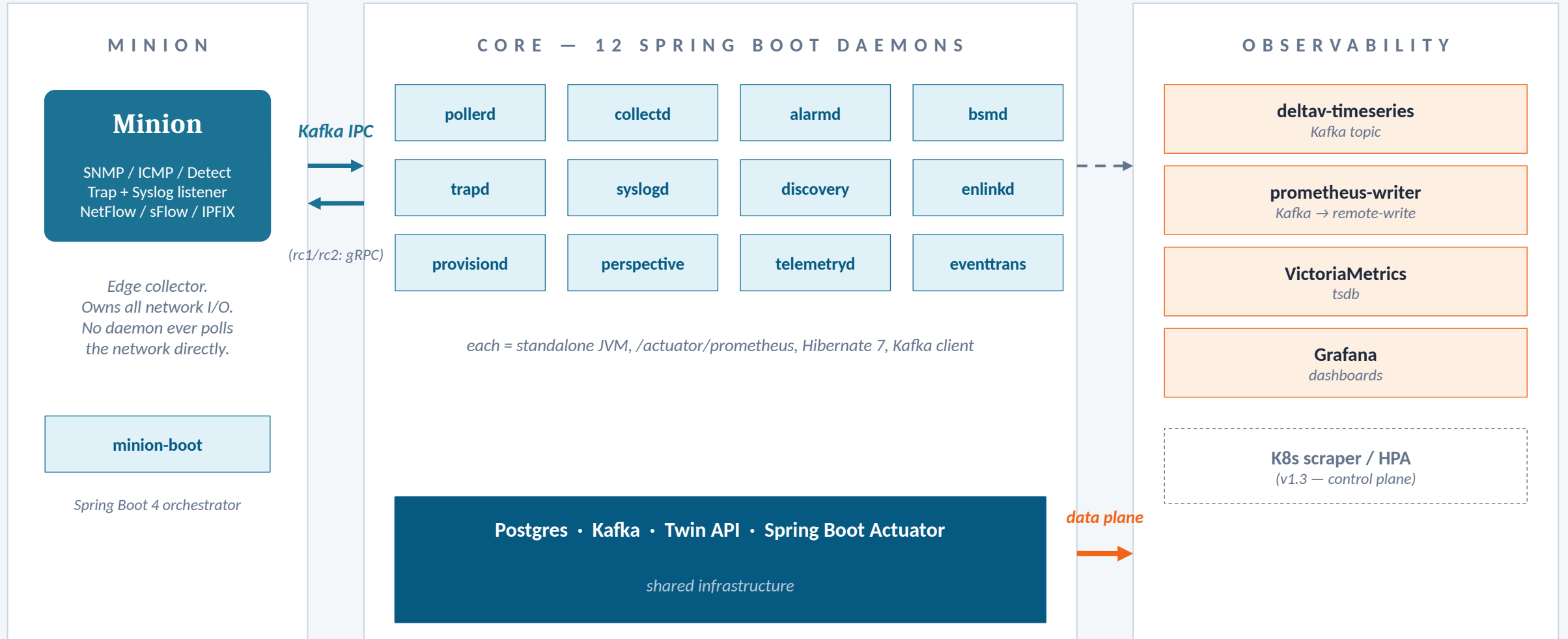
- Karaf-hosted OSGi bundles
- Monolithic opennms-services.jar
- Newts / Cassandra for time-series
- JMX MBeans for instrumentation
- ActiveMQ + ServiceMix bundles for IPC
- Hibernate 3.x, javax.persistence
- Spring 4.2.x (OpenNMS-patched)

What delta-v looks like now

- 13 standalone Spring Boot 4 daemons (Java 21)
- Hibernate 7, jakarta.persistence
- Karaf, OSGi, opennms-services — all gone
- Kafka for inter-daemon events + Minion IPC
- VictoriaMetrics + Grafana for time-series
- Micrometer + /actuator/prometheus for metrics
- Horizon code consumed as pre-built JARs

TAKEAWAY Domain model and DAOs are recognizably OpenNMS. Everything else is a clean-room refactor.

The big picture



Two planes, two purposes

CONTROL PLANE

Application observability

How the daemons themselves are doing

Metric prefix: `deltav_<daemon>_*` (native Micrometer)

Surface: `/actuator/prometheus` on each daemon

Consumer: future K8s scraper / HPA-Adapter

Examples: *polls dispatched, alarm-create rate, RPC duration p99, queue depth*

Drives autoscaling decisions, not user dashboards.

DATA PLANE

Monitored network

How the network being monitored is doing

Metric prefix: `opennms_*` (translated from CollectionSet)

Pipeline: *TimeseriesBatch protobuf → Kafka → VM*

Consumer: *Grafana, future thresholder, replaces Newts*

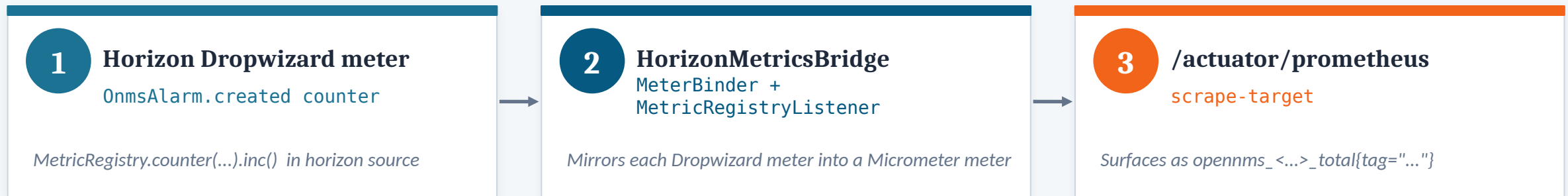
Examples: *ICMP response time, iflnOctets, BGP peer state, CPU load*

Drives operator dashboards and SLOs.

RULE These never share a TSDB. VictoriaMetrics holds data plane only.

The horizon-metric bridge

Horizon code (now consumed as pre-built JARs from [pbrane/delta-v-horizon](#)) still emits its meters via Dropwizard `MetricRegistry`. Spring Boot speaks Micrometer. Bridge between the two without editing horizon source.



And in parallel: the `deltav_*` native side

`deltav_<daemon>_*` counters are Micrometer-native — added directly in delta-v code via 5 reusable hook idioms (next slide). They live alongside the bridged `opennms_*` meters, so the same daemon's `/actuator/prometheus` emits both. The two-prefix convention lets ops tell which side a meter came from at a glance.

Five idioms across thirteen daemons

The Boot4 daemons all wrap horizon classes. Each instrumentation hook fits one of these five patterns. Discovered empirically while shipping v1.2.0-beta2; codified afterward as a reusable cheat sheet.

1

No-op listener replace

alarmd

Replace anonymous no-op Listener {} with a named CountingX impl.

Doubles as a null-op stub fix.

2

Lambda wrap inline

bsmd

Wrap a @Bean-returned lambda with Timer.record() + counters in the @Bean method itself.

Zero new files.

3

Subclass horizon class

pollerd, trapd, syslogd

extends horizon class, override entry method, increment counters, call super.

Best when the public seam is clean.

4

Decorator on choke point

enlinkd, provisiond

When N implementations all funnel through one interface, wrap the interface once.

1 wrap covers all N.

5

BeanPostProcessor

minion-boot

Auto-wrap every bean of type X. For open-ended sets (RpcModules, listeners).

Future-proof.

MEMORY `feedback_daemon_instrumentation_patterns` — bytecode-audit horizon's class first, then pick the idiom. No 6th idiom needed across all 13.

What the data plane carries today

Producers → **deltav-timeseries** Kafka topic

collectd

v1.1.x

SNMP CollectionSets per scheduled poll

pollerd

v1.2.0-beta2 (just shipped)

ICMP / TCP / HTTP response-time samples

perspectivepollerd

post-beta2

(Phase 3 deferred)

minion-snmp-collector

v1.3+

(future)

Live evidence — Apr 25 smoke

```
$ curl -sG 'localhost:18428/api/v1/query'
--data-urlencode 'query={producer="pollerd"}'

{
  "status": "success",
  "data": { "resultType": "vector",
    "result": [{
      "metric": {
        "__name__": "opennms_response_time_response",
        "producer": "pollerd",
        "location": "l8opensim-lab",
        "node_label": "lab-01-core-rtr-01",
        "foreign_source": "l8opensim-lab",
        "collection_package": "ICMP",
        "resource_type": "monitoredService"
      },
      "value": [ 1745604821, "3.2" ]
    }] } }
```

INVARIANT Same metric name across producers; *producer*= label distinguishes them.

Familiar / unfamiliar

Layer	Status in delta-v	Notes for you
Domain model (OnmsNode, OnmsAlarm, ...)	Same classes, jakarta.persistence	Annotations updated; semantics unchanged
DAOs	Same interfaces, JPA-based impls	Hibernate 7 / Jakarta; HQL queries preserved
Daemon classes (Pollerd, Alarmd, ...)	Used as horizon JARs; subclassed in Boot4	AbstractServiceDaemon → SmartLifecycle adapter
EventBuilder, EventForwarder	Still the API; Kafka is the transport	No more EventBus, no more Eventd
OSGi / Karaf / Blueprint	Gone	Spring Boot 4 @Configuration replaces blueprint XML
JMX MBeans for instrumentation	Largely replaced	Spring Boot Actuator + Micrometer
Newts / Cassandra	Gone	VictoriaMetrics + Kafka pipeline
ActiveMQ / ServiceMix / camel	Going (rc1/rc2)	gRPC migration removes the last refs
Spring 4.2.x (OpenNMS-patched)	Gone	Spring 7 / Spring Boot 4.0.3, Java 21

Quick scan: if a class lives in `org.opennms.netmgt.*`, treat it as familiar. If it lives in `org.deltav.*`, it's new code we own.

Where v1.2 closes, where v1.3 begins

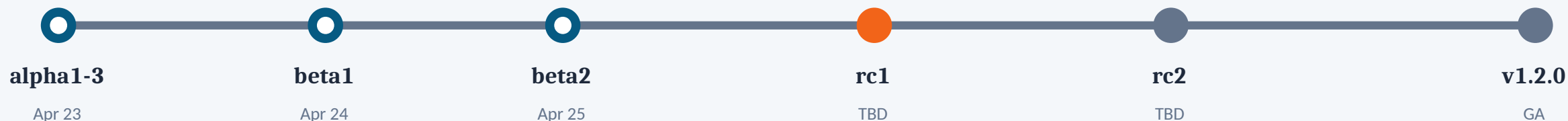
self-contained images

horizon-metric bridge

domain metrics + Pollerd P3

Minion gRPC midpoint

gRPC complete · Kafka removed



rc1 / rc2 — Minion gRPC migration

Replace Kafka-based RPC + Sink with gRPC, *fronted by Spring Cloud Gateway*.

Heartbeat-first migration in rc1 (parallel Kafka + gRPC). rc2 removes Kafka IPC entirely, purges 32 ActiveMQ + ServiceMix bundles, retires the Default Minion location.

Gateway becomes the single ingress for any future HTTP UI too.

v1.3 — K8s operator + HPA

The control plane built in v1.2 finally gets a consumer.

An operator scrapes `deltav_<daemon>_*` from each pod's actuator and feeds HPA v2. **Autoscale on queue depth and RPC p99**, not CPU and memory (which lag the actual signal).

Replaces Nephron with Spring Cloud Stream consumers; YAML config migration.

Concrete observability surfaces with room

OBSERVABILITY

Define HPA-grade metric semantics

We have ~80 `deltav_*` counters across 13 daemons. Which 5 should HPA-Adapter expose to a K8s scaling policy? What's the right histogram bucket choice for RPC duration? What thresholds matter and which are noise?

Your call: bring the operator perspective to a metric set that's currently a developer's-eye view.

NEPHRON REPLACEMENT

Spring Cloud Stream Thresholder

The kafka-timeseries pipeline lands in VictoriaMetrics today; nothing yet consumes it for thresholding. Design as an SCS consumer with standard-deviation rules, replacing horizon's inline `ThresholdingVisitor`.

Greenfield design space; existing `TimeseriesBatch` protobuf is the input contract.

TRACING

OpenTelemetry on the gRPC migration

Minion → Core gRPC migration in v1.2-rc1/rc2 gives us per-call tracing for free. The deployment-layer question (Tempo? Jaeger? Grafana Cloud?) is open. So is the span model: which propagation context flows through `TimeseriesBatch`?

Lands on a v1.2-rc1 schedule, so the timing is right.